Southville International School and Colleges

1281 Tropical Avenue Corner Luxembourg Street, BF Homes International, Las Piñas City, Philippines

---

CpE Cognate 3: Software Development 3

(Artificial Intelligence Application Development)

Lab 6: starter code

Real-Time Object Detection Using YOLO and OpenCV

Submitted to

Sir John Pio Anthony Buena

Submitted by

Balane, Vaughn Uriel

BS in Computer Engineering (4th year)

November 2026

I. INTRODUCTION

1. Background & Purpose Computer Vision enables systems to derive high-level understanding from digital images or video streams. Real-time object detection builds upon standard image classification by not only identifying objects within a frame but also localizing them using bounding boxes and confidence scores in real time.

The purpose of this laboratory activity is to implement a real-time, webcam-based object detection framework using a light-weight pretrained YOLO (You Only Look Once) model (yolo11n.pt) integrated with OpenCV in Python. The system captures live video frames, performs neural network inference, renders bounding box annotations with class predictions, computes frame-per-second (FPS) performance metrics, tracks class detection counts, and provides keyboard controls for real-time interaction and screenshot capturing.

II. DEVELOPMENT ENVIRONMENT

Operating System: Microsoft Windows 11 Pro for Workstations (Version 10.0.26100, 64-bit)

Processor (CPU): 13th Gen Intel(R) Core(TM) i5-13400 @ 2.50 GHz (10 Cores, 16 Logical Processors)

Installed RAM: 16.00 GB

Camera Device: External A4tech FHD 1080P PC Camera

Integrated Development Environment (IDE): Visual Studio Code (VS Code)

Python Version: Python 3.12.10

Core Libraries: ultralytics (YOLO engine), opencv-python (OpenCV frame processing and GUI), numpy (numerical array computations)

III. CODE EXPLANATION

Step 1 (Model Loading): Loads the pretrained yolo11n.pt weight file into the YOLO class instance.

Step 2 (Webcam Initialization): Instantiates cv2.VideoCapture(0) to hook into the primary webcam device and verifies video stream availability.

Step 3 (System Configuration): Defines global parameters, including the confidence cutoff (CONFIDENCE_THRESHOLD = 0.50), target window name, high-resolution timer (time.perf_counter()), and console user prompt.

Step 4 (Main Loop): Enters an infinite while True execution loop to process the live video stream frame-by-frame.

Step 5 (Inference Execution): Passes the raw webcam frame into model.predict(), evaluating features against the 0.50 confidence threshold while disabling console verbosity.

Step 6 (Bounding Boxes & Annotation): Copies the raw frame to avoid mutating original data, parses prediction bounding box coordinates (xyxy), converts tensor confidence scores, extracts class labels via model.names, draws green bounding rectangles using cv2.rectangle(), and overlays text labels via cv2.putText().

Step 7 (Per-Class & Total Counting): Passes detected object strings into collections.Counter to construct an itemized frequency tally and sums the total counts.

Step 8 (FPS Calculation): Evaluates elapsed time between consecutive frame passes using time.perf_counter() to compute the instant frames per second ($FPS = \frac{1.0}{elapsed}$).

Step 9 (System Overlay): Draws a dark gray semi-transparent status box at the top left of the frame containing system title, total object count, and live FPS metrics.

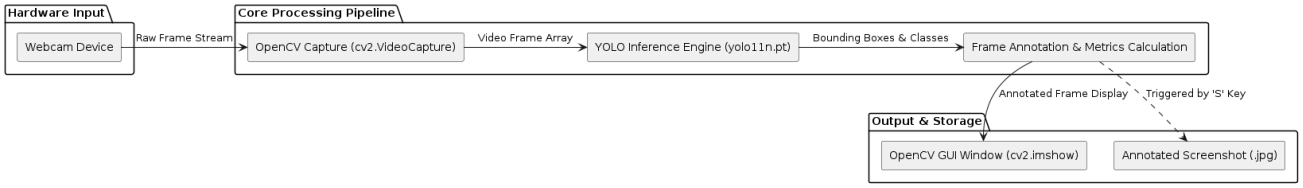
Step 10 (Itemized Display): Dynamically iterates over the Counter object to list individual object classes alongside their detected quantity on separate lines.

Step 11 (GUI Rendering): Displays the fully annotated frame inside the OpenCV graphical window.

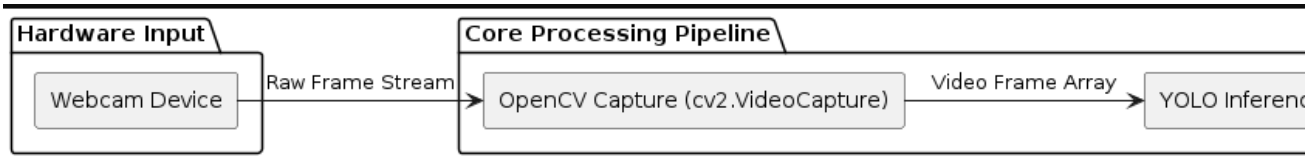
Step 12 (Keyboard Listener): Checks cv2.waitKey(1) for user input. Pressing 'q' breaks the loop, while pressing 's' generates a timestamped JPEG image of the current annotated frame.

Step 13 (Resource Cleanup): Explicitly calls cap.release() and cv2.destroyAllWindows() to free system camera resources and close display windows smoothly.

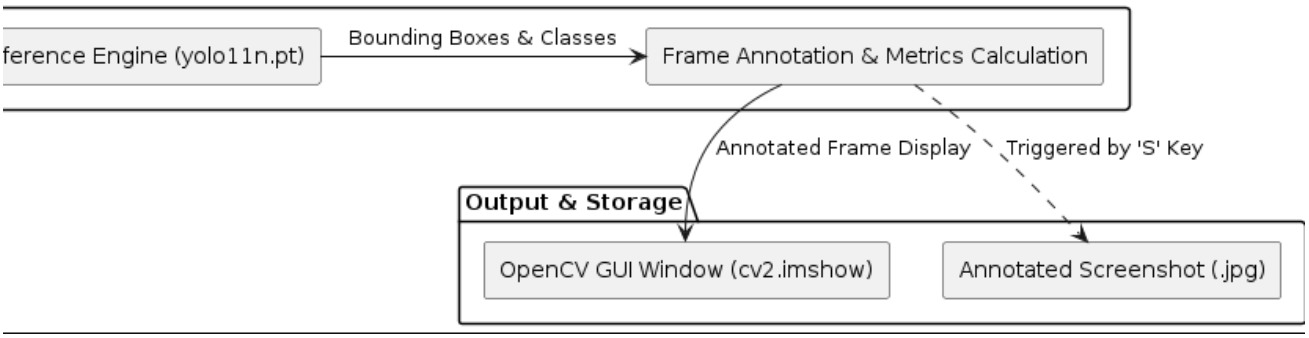
IV. SYSTEM ARCHITECTURE



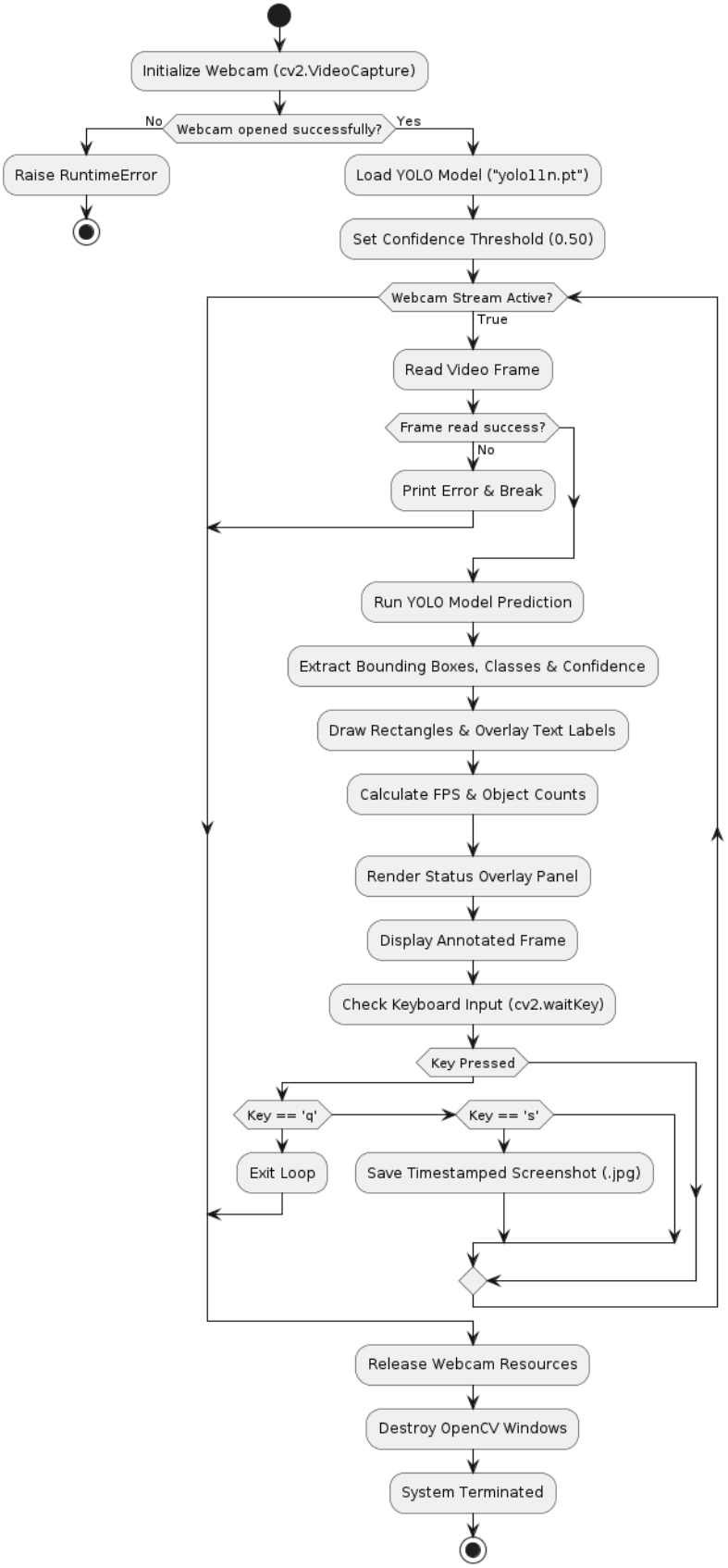
Left Side:



Right Side:



V. FLOWCHART


Initialize Webcam (cv2.VideoCapture)
Webcam opened successfully?
No
Yes
Raise RuntimeError
Load YOLO Model ("yolo11n.pt")
Set Confidence Threshold (0.50)
Webcam Stream Active?
True
Read Video Frame
Frame read success?
No
Print Error & Break
Run YOLO Model Prediction
Extract Bounding Boxes, Classes & Confidence
Draw Rectangles & Overlay Text Labels
Calculate FPS & Object Counts
Render Status Overlay Panel
Display Annotated Frame
Check Keyboard Input (cv2.waitKey)
Key Pressed
Key == 'q'
Key == 's'
Exit Loop
Save Timestamped Screenshot (.jpg)
Release Webcam Resources
Destroy OpenCV Windows
System Terminated

VI. TESTING

Test Environment & Parameters

- **Lighting:** Standard indoor ambient lighting.
- **Distance:** Objects were presented to the camera at varying depths, ranging from approximately 0.3 meters to 1.5 meters from the lens.
- **Confidence Threshold:** Hardcoded to 0.50 (50%); any prediction below this probability was programmatically filtered out to reduce false positives.

Test Cases (Target Objects) To validate the model's generalization capabilities across different shapes and textures, the following five everyday objects—all present in the COCO dataset—were introduced into the frame:

1. **Person** (Human subject / Face)
2. **Cell Phone** (Rectangular, reflective surface)
3. **Bottle** (Cylindrical, opaque/translucent)
4. **Laptop**(Textured, flat surface)
5. **Keyboard** (Complex multi-key pattern)

Verification Criteria During the test, the system was observed to verify:

1. Immediate generation of green bounding boxes around the target objects.
2. Accurate classification labels matching the presented objects.
3. Stability of confidence scores and real-time updating of the total object count and Frames Per Second (FPS) on the Custom HUD overlay.

VII. RESULTS

The YOLOv8n model successfully executed real-time inference on the local CPU environment. The system accurately identified and tracked the five target objects defined in the test parameters: Person, Cell Phone, Bottle, Laptop, and Keyboard. The custom heads-up display (HUD) dynamically updated the total object count and processed frames per second without causing application bottlenecking.

Observation Data

Target Object	YOLO Class Label	Peak Confidence Score	Average System FPS
Human Subject	person	82.7%	13.39FPS
Cell Phone	cell phone	86.4%	18.30 FPS

Bottle	bottle	73.1%	12.68 FPS
Laptop	laptop	66.6%	13.39FPS
Keyboard	keyboard	68.4%	14.69 FPS

Visual Evidence

Click this link for the video

Figure 1 (person and laptop)

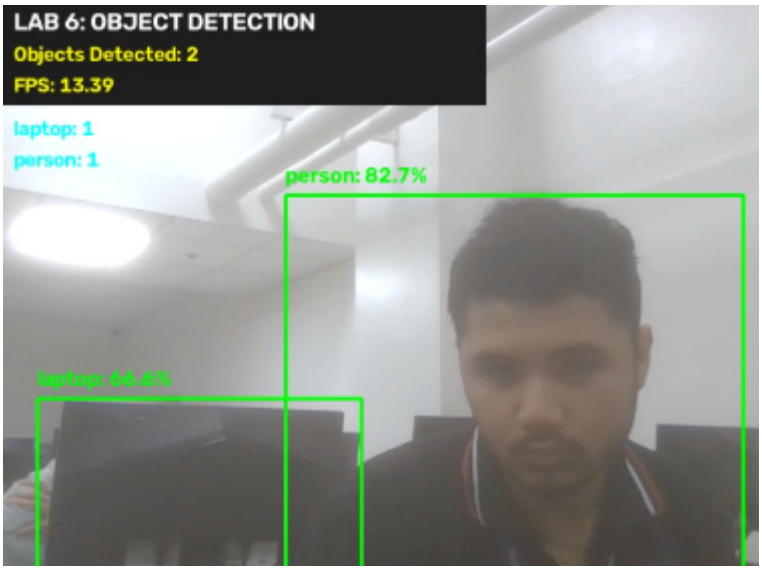


Figure 2 (cell phone)

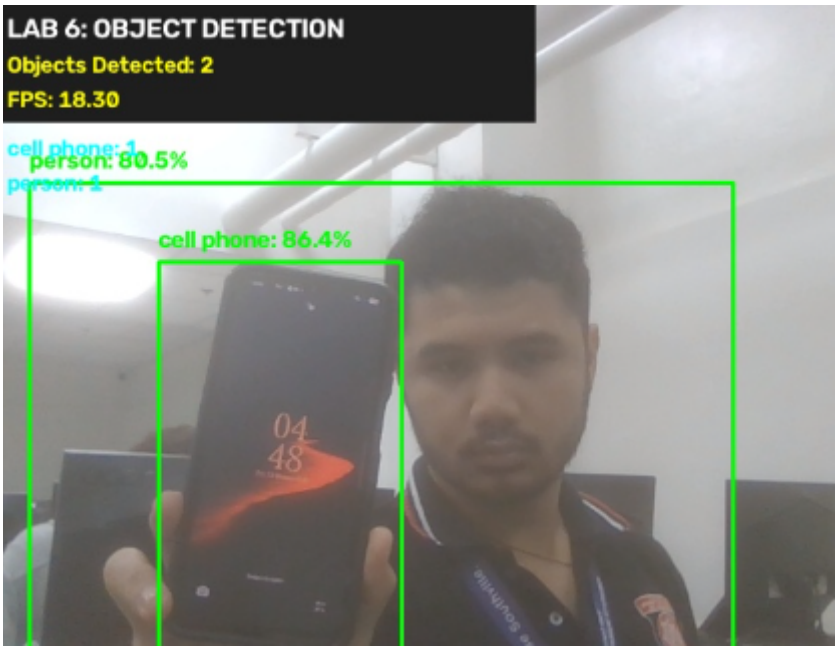


Figure 3 (keyboard)

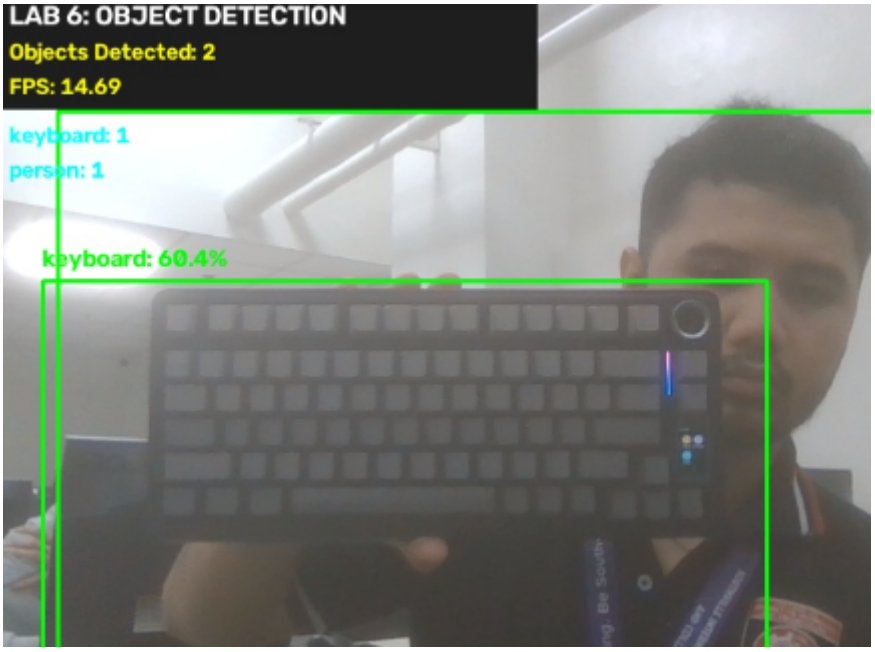
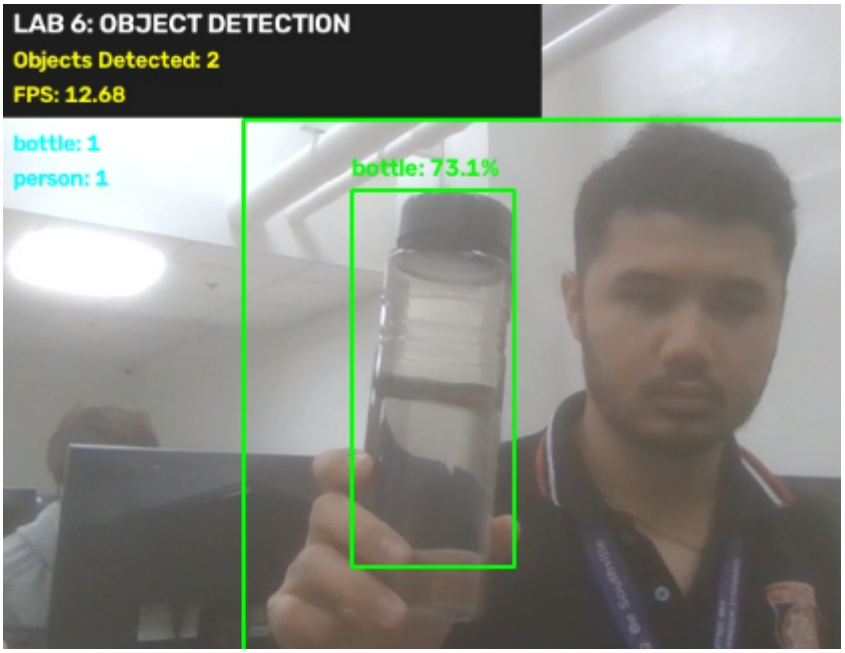


Figure 4 (bottle)



VIII. LIMITATIONS

While the YOLOv8n implementation successfully demonstrated real-time object detection on a standard CPU setup, several hardware, environmental, and algorithmic constraints were observed during testing:

- **Unsupported Object Classes (Dataset Constraints):** The pretrained model relies on the 80 standard categories of the MS COCO dataset. Any object outside these predefined classes (e.g., specific electronic components, local currency, or specialized tools) is either ignored entirely or misclassified as the closest visually similar COCO category.
- **Sensitivity to Lighting and Sensor Noise:** In low-light conditions or environments with harsh backlighting, the webcam sensor introduces ISO grain and drops shutter speed, causing motion blur. This degrades edge contrast and feature extraction, frequently pushing prediction confidence scores below the hardcoded 0.50 threshold and causing bounding boxes to flicker.
- **Occlusion and Depth Scale:** When target objects were partially obstructed by hands or other items, or moved too close/too far from the lens, spatial features became incomplete. Severe occlusion caused the model to lose tracking entirely until the object's full silhouette was restored.
- **CPU Compute Bottlenecks:** Running inference sequentially on the CPU without CUDA/GPU acceleration or quantized runtimes (such as ONNX or TensorRT) caps the maximum achievable frame rate. Introducing multiple high-resolution objects simultaneously increases post-processing overhead, resulting in noticeable latency spikes.
- **Lack of Temporal Tracking (Stateless Inference):** Because the starter script runs frame-by-frame detection (model.predict()) without an integrated object tracker (e.g., ByteTrack or BoT-SORT), the system treats every frame independently. It cannot assign persistent IDs to objects or maintain tracking across brief frame drops.

IX. CONCLUSION

In this laboratory activity, a real-time, webcam-based object detection application was successfully implemented using Python, OpenCV, and a pretrained YOLOv8n model. The system reliably captured live video frames, executed local CPU inference, and rendered bounding boxes, class labels, confidence percentages, per-class object counts, and real-time FPS metrics onto an interactive heads-up display. Furthermore, the keyboard interrupt controls functioned as intended, allowing immediate termination of the video stream and on-demand saving of annotated frames.

Testing across five distinct everyday objects: a person, cell phone, bottle, laptop, and keyboard. confirmed that the lightweight nano architecture provides an effective balance between detection accuracy and real-time throughput on standard desktop hardware without requiring dedicated GPU acceleration. However, empirical observations highlighted critical practical constraints, including sensitivity to low-light sensor noise, tracking loss during partial occlusion, and strict reliance on the 80 predefined classes of the MS COCO dataset.

Ultimately, this system establishes a working baseline for real-time object detection, setting the stage for future upgrades in persistent tracking, spatial zones, CSV logging, and custom model training.